# Learning from the Self-future: On-policy Self-distillation for dLLMs

*Yifu Luo, Zeyu Chen, Haoyu Wang, Xinhao Hu, Yuxuan Zhang, Zhizhou Sha, Shiwei Liu*

ABSTRACT

*On-policy self-distillation (OPSD) has proven effective for post-training large language models (LLMs), yet its application to diffusion LLMs (dLLMs) remains unexplored. Existing OPSD methods are inherently autoregressive-centric. They inject privileged information via left-to-right prefix conditioning with token-level divergence supervision, a design that fundamentally conflicts with the arbitraryorder generation of dLLMs. We introduce d-OPSD, the first OPSD framework tailored for dLLMs. Our approach makes two core contributions. First, we reframe self-teacher construction by using self-generated answers as suffix conditioning, enabling the student model to learn from "self future-experience" rather than privileged prefixes. Second, we shift supervision from token-level to step-level, aligning training with the iterative denoising process of dLLMs. Experiments across four reasoning benchmarks show that d-OPSD consistently outperforms RLVR and SFT baselines with superior sample efficiency, requiring only around 10% of the optimization steps by RLVR and opening a promising pathway for dLLM posttraining. The code is available at https://github.com/xingzhejun/d-OPSD.*

## About this document

This is a Reviewed Preprint: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page exactly as published by its authors — llmXive re-typesets nothing and claims no authorship.
Provenance. Ingested by llmXive on 2026-07-02 — scraped from arXiv; via llmXive dashboard, GitHub issue #336; submitted by github-actions[bot].
Source. `https://arxiv.org/abs/2606.18195`
About llmXive. llmXive is an automated platform for scientific discovery in which specialist LLM agents advance ideas from a one-paragraph brainstorm to a peer-reviewed paper. Learn more at `https://github.com/ContextLab/llmXive`.
Automated review. See the accompanying `peer-review-llmxive.pdf` for the full reviewer report.
llmXive follow-up. A separate llmXive study building on this work: `PROJ-847-llmxive-follow-up-extending-learning-fro`.

# Learning from the Self-future: On-policy Self-distillation for dLLMs

**Yifu Luo** [1,†], **Zeyu Chen** [2,†], **Haoyu Wang** [3], **Xinhao Hu** [1],
**Yuxuan Zhang** [4], **Zhizhou Sha** [5], **Shiwei Liu** [6,7,8,*]
[†]Equal Contribution

[1]Tsinghua University [2]Technical University of Munich [3]Nanyang Technological University
[4]University of British Columbia [5]University of Texas at Austin [6]ELLIS Institute Tübingen
[7]Max Planck Institute for Intelligent Systems [8]Tübingen AI Center

## Abstract

On-policy self-distillation (OPSD) has proven effective for post-training large language models (LLMs), yet its application to diffusion LLMs (dLLMs) remains unexplored. Existing OPSD methods are inherently autoregressive-centric. They inject privileged information via left-to-right prefix conditioning with token-level divergence supervision, a design that fundamentally conflicts with the arbitrary-order generation of dLLMs. We introduce d-OPSD, the first OPSD framework tailored for dLLMs. Our approach makes two core contributions. First, we reframe self-teacher construction by using self-generated answers as suffix conditioning, enabling the student model to learn from "self future-experience" rather than privileged prefixes. Second, we shift supervision from token-level to step-level, aligning training with the iterative denoising process of dLLMs. Experiments across four reasoning benchmarks show that d-OPSD consistently outperforms RLVR and SFT baselines with superior sample efficiency, requiring only around $10\%$ of the optimization steps by RLVR and opening a promising pathway for dLLM post-training. The code is available at `https://github.com/xingzhejun/d-OPSD`.

## 1 Introduction

On-policy distillation (OPD) [Agarwal et al., 2024, Yang et al., 2025, Lu and Lab, 2025, Li et al., 2026], where a student model samples its own trajectories while a stronger teacher model provides dense token-level supervision, has recently emerged as a highly effective paradigm for Large Language models (LLMs) post-training, offering significant advantages over Reinforcement Learning with Verifiable Rewards (RLVR) (e.g., GRPO [Guo et al., 2025] and supervised fine-tuning (SFT). Compared to RLVR, OPD provides dense token-level supervision from a teacher, over-

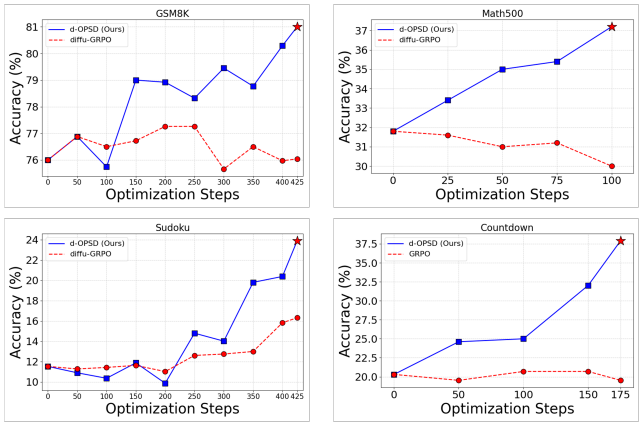


Figure 1: The reasoning performance and sample efficiency comparisons between the RLVR baseline (diffu-GRPO[Zhao et al., 2025]) and our approach, d-OPSD.

*Corresponding Author.

coming the bottleneck of sparse outcome rewards. Compared to SFT, OPD utilizes generations sampled from the student itself, thereby preventing exposure bias [Bengio et al., 2015]. However, OPD relies heavily on a stronger teacher model, which is often impractical in many settings. To address this, recent works [Zhao et al., 2026, Hübotter et al., 2026, Shenfeld et al., 2026] have extended OPD to on-policy self-distillation (OPSD), where a single model serves as its own teacher given teacher-specific privileged information, demonstrating a powerful framework for self-improvement.

Concurrently, diffusion large language models (dLLMs) [Ou et al., 2024, Nie et al., 2025, Ye et al., 2025, Cheng et al., 2025, Bie et al., 2025] have demonstrated strong potential as an alternative to autoregressive (AR) LLMs [Jaech et al., 2024, Xiao et al., 2026]. By modeling language generation as an iterative denoising process, dLLMs bypass the strict left-to-right dependency of AR models, unlocking unique advantages such as arbitrary-order generation and speed-up inference [Khanna et al., 2025, Song et al., 2025, Wu et al., 2025].

While recent works [Zhao et al., 2025, Tang et al., 2025, Xie et al., 2025] has successfully applied RLVR to dLLMs demonstrating that their reasoning ability can be enhanced by post-training, OPSD for dLLMs remains largely unexplored in this context. Meanwhile, as shown in Figure 2, existing OPSD approaches for AR models follow a standard paradigm for self-teacher construction, where privileged information (e.g., reference solutions) is simply appended to the prompt, and teacher-student divergence supervision is calculated at the token level. Given that dLLMs exhibit fundamental different features from AR LLMs, we investigate the following two questions in this paper:

---

**Question:** Is there a better OPSD formulation tailored specifically for dLLMs?
**Answer:** Yes. Both the self-teacher construction and the level of divergence supervision can be optimized for dLLMs, as shown in Figure 2.

**Question:** Does OPSD outperform RLVR in enhancing the reasoning ability of dLLMs?
**Answer:** Yes. It achieves superior results in both reasoning performance and sample efficiency, as shown in Figure 1.

---

First, we identify that the self-teacher construction mentioned above is suboptimal for dLLMs. Appending privileged information to the prompt is inherently designed for AR models, because they are constrained to left-to-right generation where only prefix conditioning $p(\text{suffix}|\text{prefix})$ is available. In contrast, dLLMs generate sequences non-autoregressively, which allows us to incorporate privileged information as a suffix context condition. More importantly, this feature enables us to shift the content of privileged information from static reference solutions to the model's self-generated answers, adhering closer to the on-policy nature. As shown in Figure 2, the $p(\text{prefix}|\text{suffix})$ capability of dLLMs allows us to use self-generated answers as a suffix conditional posterior for privileged information. This guides the student to learn from "self future-experience", which is similar to human inspiration that we always daydream if we could go back to 10 years ago knowing what happened next. A key advantage of our teacher construction is that it provides more new knowledge (thinking patterns) to transfer to the student, a claim we empirically discuss in Section 4.3.

Second, token-level divergence supervision is not suitable for dLLMs either. While AR models natively rely on next-token prediction, dLLMs predict all masked tokens simultaneously at each denoising step, but only keep part of them while remasking others. Consequently, token-level supervision designed for AR models becomes incompatible. Instead, as each denoising step can be viewed as an independent markov transition, step-level divergence serves as a nature choice for dLLMs OPSD. By shifting the dense supervision from the token-level to the step-level, we closely align the OPSD objective with the iterative denoising nature of dLLMs.

Building on these insights, we propose **diffusion On-Policy Self-distillation (d-OPSD)**, a novel OPSD framework specifically designed for dLLMs to drive self-improvement. To the best of our knowledge, this represents the first application of OPSD to dLLMs. In our approach, the student samples its own trajectories, while the self-teacher is constructed using self-generated answers as suffix privileged information. By applying step-level divergence, the student effectively learns from its "self future-experience". Extensive experiments across four reasoning tasks demonstrate that our approach consistently outperforms RLVR and SFT baselines with superior reasoning performance and sample efficiency, as highlighted in Figure 1.

Our contributions are summarized as follows:

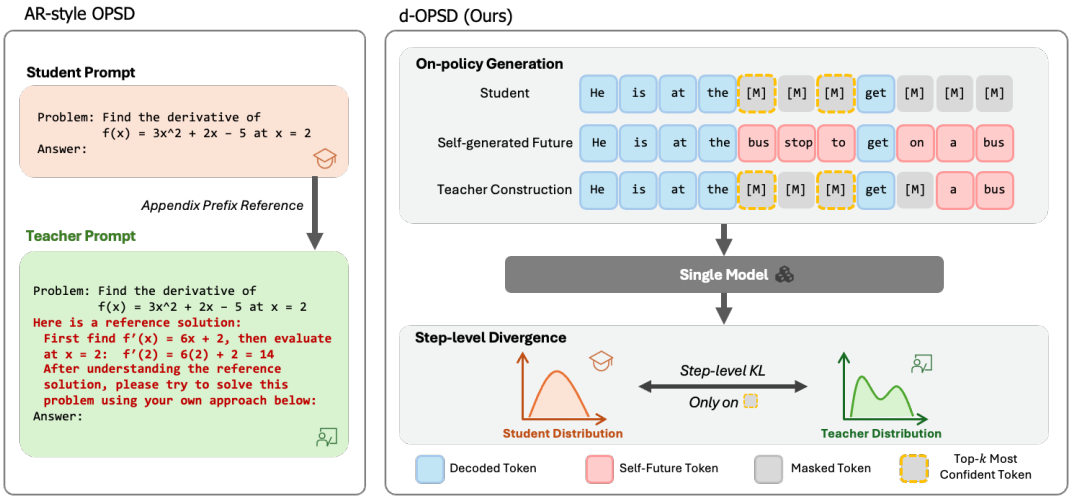


Figure 2: The framework of our approach, d-OPSD. It leverages self-generated answers as suffix privileged information to construct the self-teacher, and uses step-level divergence to guide the student learn from the "self future-experience".

- We identify that existing OPSD formulations are suboptimal for dLLMs. To bridge this gap, we introduce a novel self-teacher construction that utilizes self-generated answers as a suffix conditional posterior for privileged information, and we shift the dense divergence supervision from the token-level to the step-level.
- We are the first to introduce OPSD to dLLMs. We propose d-OPSD, a novel OPSD framework tailored for dLLMs to drive self-improvement. It enables a single model to act as both teacher and student, leveraging self-generated "future" as privileged information to provide dense step-level supervision over the student trajectories.
- We conduct extensive experiments across four reasoning tasks, demonstrating that our approach achieves both superior reasoning performance and sample efficiency compared to RLVR and SFT baselines. Furthermore, we empirically analyze the impact of different settings, paving the way for future advances in this field.

## 2 Preliminaries

### 2.1 Diffusion Large Language Models

In this subsection, we briefly review the training and inference paradigms of dLLMs. During training, dLLMs define a forward process that gradually corrupts a clean input by replacing its tokens with a special `mask` token. Given a prompt $x$ and a clean response $y_0 = \{y_0^1, y_0^2, \cdots, y_0^L\}$, the forward process at step $0 < t \leq T$ can be expressed as:

$$q(y_t|y_0, x) = \prod_{i=1}^{L} q(y_t^i|y_0^i, x) \quad \text{and} \quad q_t(y_t^i|y_0^i, x) = \begin{cases} \frac{T-t}{T}, & y_t^i = y_0^i, \\ \frac{t}{T}, & y_t^i = \texttt{mask}, \end{cases} \tag{1}$$

where L is the sequence length, and the superscript $i$ refers to the token position.

In this work, we primarily focus on the reverse inference process of dLLMs. Given a prompt $x$ and a trained model $p_\theta$, inference is formulated as a $T$-step iterative denoising process, from a fully masked sequence $y_T = \{\texttt{mask}\}^L$ to a clean response $y_0$. At each denoising step $t$, the model first computes the distribution for all tokens:

$$\mathcal{P}_t^i = p_\theta(y^i|y_t, x), \quad 1 \leq i \leq L. \tag{2}$$

For the top-$k$ most confident predictions among the currently masked positions, they are sampled and revealed. The remaining masked positions are kept masked as `mask` and to form $y_{t-1}$. After $T$ steps, all masked tokens are revealed, yielding the final response $y_0$. Additional preliminaries about block-diffusion, a common-used inference strategy, are provided in Section A.1.

### 2.2 On-policy Distillation

OPD transfers knowledge from a stronger teacher model $p_T$ to a weaker student model $p_\theta$ by enforcing dense supervision over trajectories sampled by the student. For AR models, given a prompt $x$, the student samples a response $y = \{y^1, y^2, \cdots, y^L\}$. Using the AR factorization, the learning objective is to minimize the token-level KL between the teacher's and the student's next-token distributions:

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_x \left[ \sum_{i=1}^{L} \mathcal{D}_{\text{KL}} \left( p_\theta \left( \cdot | y^{<i}, x \right) || \left( p_T \left( \cdot | y^{<i}, x \right) \right) \right) \right], \tag{3}$$

where $p(\cdot|y^{<i}, x)$ denotes the distribution over the next token $y^i$. While we use reverse KL, forward KL and other distribution divergence measures like generalized Jensen-Shannon divergence [Agarwal et al., 2024] can also be employed.

Recent advances have extended OPD to OPSD, where the student and teacher are instantiated from the same model, denoted as $p_\theta$. The difference lies entirely in their conditioning contexts. For AR models, privileged information $y^*$, such as reference solutions [Zhao et al., 2026] or environment feedback [Hübotter et al., 2026], is appended to the original prompt $x$ to construct a teacher-specific prompt $x^* = x + y^*$. Thus, the teacher distribution is:

$$p_T = p_\theta \left( \cdot | y^{<i}, x, y^* \right) = p_\theta \left( \cdot | y^{<i}, x^* \right). \tag{4}$$

Consequently, the learning objective in Equation (3) adapts into the following:

$$\mathcal{L}_{\text{OPSD}}(\theta) = \mathbb{E}_x \left[ \sum_{i=1}^{L} \mathcal{D}_{\text{KL}} \left( p_\theta \left( \cdot | y^{<i}, x \right) || \left( p_\theta \left( \cdot | y^{<i}, x^* \right) \right) \right) \right]. \tag{5}$$

In this setup, both the teacher and student share the same model but differ only in the conditioning contexts, and the response is solely generated from the student. While OPSD achieves comparable performance to RLVR with superior sample efficiency for AR models, adapting this formulation to dLLMs presents fundamental challenges. First, the arbitrary-order generation of dLLMs provides an alterative for injecting privileged information, which better aligns with on-policy nature (Section 3.1). Second, token-level divergence supervision is incompatible with dLLMs as next-token prediction is not factorized. Instead, step-level divergence supervision must be adopted(Section 3.2).

## 3 Methods

### 3.1 Teacher Construction: Learning from the Self-future

In this section, we describe how we utilize the student's self-generated "future" answers as privileged information for the teacher, which adheres closer to dLLMs and on-policy nature. While AR models are constrained to left-to-right generation with only $p(\text{suffix}|\text{prefix})$ available, dLLMs possess the bidirectional capability to model suffix conditioning $p(\text{prefix}|\text{suffix})$. As shown in Figure 2, our core insight is that after sampling a complete trajectory from the student, we can partially reveal this self-generated subsequent trajectory to the teacher as privileged information.

Specifically, We instantiate both the teacher and student distributions from the same dLLM $p_\theta$ by varying the conditioning inputs. Given a prompt $x$, the student first samples a trajectory from $p_\theta$:

$$Y = \{y_T, y_{T-1}, \cdots, y_0\} \sim p_\theta(\cdot|x), \tag{6}$$

where $y_T = \{\texttt{mask}\}^L$ is a fully masked sequence, $y_0$ is the final response, $T$ refers to the total number of denoising steps, and $L$ denotes the sequence length. At each denoising step $0 < t \leq T$, the student input is simply the current noisy sequence:

$$y_{\text{student},t} = y_t. \tag{7}$$

Conversely, the teacher input is constructed by selectively revealing tokens from the final generated response $y_0$:

$$y^i_{\text{teacher},t} = \begin{cases} y^i_0, & \text{if } i \in \mathcal{S}_t, \\ y^i_t, & \text{otherwise}, \end{cases} \tag{8}$$

where $\mathcal{S}_t \subset \{1, 2, \cdots, L\}$ is the revealing subset of indices randomly selected with a fixed retaining ratio $\rho_{\text{teacher}}$ from the currently masked positions. Thus, both the student and teacher share the same model $p(\theta)$, but the teacher benefits from the self-generated "future" trajectory. An illustration example of our teacher construction is provided in Section B.

This construction seamlessly aligns with on-policy and dLLMs nature. First, all data is generated by the student. Second, the construction in Equation (7) and Equation (8) yields distributions $p_\theta(\cdot|y_{\text{student},t}, x)$ and $p_\theta(\cdot|y_{\text{teacher},t}, x)$, which enable a direct step-level divergence supervision, which we introduce in the next subsection. Note that $p(\cdot|y_t, x)$ here denotes distribution for the next step.

### 3.2 Step-level Divergence Supervision

Unlike AR models, which natively employ token-level supervision via next-token prediction, dLLMs decode sequences via next-step prediction. At each denoising step, only the top-$k$ most confident tokens among the currently masked positions are sampled and revealed, while the remaining `mask` tokens are kept masked. While token-level supervision is incompatible, we propose step-level divergence supervision as a more natural objective for dLLMs.

Specifically, at each denoising step $t$, using the previously constructed inputs $y_{\text{student},t}$ and $y_{\text{teacher},t}$, the model first computes full-sequence distributions:

$$\begin{aligned} \mathcal{P}^i_{\text{student},t} &= p_\theta(y^i|y_{\text{student},t}, x), \quad 1 \le i \le L, \\ \mathcal{P}^i_{\text{teacher},t} &= p_\theta(y^i|y_{\text{teacher},t}, x), \quad 1 \le i \le L. \end{aligned} \tag{9}$$

Crucially, not all token positions $i$ actively participate in the state transition from $t$ to $t-1$. We focus exclusively on the top-$k$ most confident tokens among the currently masked positions, as only these tokens dictate the step-level transition. Denoting these tokens' indices as the top-$k$ subset $\mathcal{K}_t \subset \{1 \le i \le L | y^i_t = \texttt{mask}\}$ which satisfies:

$$\sum_{t=1}^{T} |\mathcal{K}_t| = L. \tag{10}$$

We then compute the step-level KL divergence over this subset:

$$\mathcal{L}_t = \frac{1}{|\mathcal{K}_t|} \sum_{i \in \mathcal{K}_t} \mathcal{D}_{\text{KL}} \left( \mathcal{P}^i_{\text{student},t} || \mathcal{P}^i_{\text{teacher},t} \right). \tag{11}$$

Note that the top-$k$ subset $\mathcal{K}_t$ can theoretically be determined from either the student distribution or teacher distribution. However, the ablation study in Section 4.4 suggests that deriving from the teacher distribution yields greater performance gains.

With the self-teacher construction and step-level divergence in place, we now possess all the essential components needed to apply OPSD to dLLMs.

### 3.3 d-OPSD: the First On-Policy Self-distillation for dLLMs

We now formally introduce our approach, **d-OPSD**. Operating with a single model $p_\theta$ severing simultaneously as student and teacher, the procedure begins with the student sampling an on-policy $T$-step trajectory $Y$ (Equation (6)) for a given prompt $x$. For each denoising step $0 < t \le T$, we construct the student input $y_{\text{student},t}$ and the teacher input $y_{\text{teacher},t}$ using Equation (7) and Equation (8). Note that the constructions are independent over steps. Finally, we minimize the following step-level learning objective across the entire on-policy trajectory:

$$\begin{aligned} \mathcal{L}_{\text{OPSD}}(\theta) =& \mathbb{E}_x \left[ \frac{1}{T} \sum_{t=1}^{T} \mathcal{L}_t \right] \\ =& \mathbb{E}_x \left[ \frac{1}{T} \sum_{t=1}^{T} \frac{1}{|\mathcal{K}_t|} \sum_{i \in \mathcal{K}_t} \mathcal{D}_{\text{KL}} \left( \mathcal{P}^i_{\text{student},t} || \mathcal{P}^i_{\text{teacher},t} \right) \right] \\ =& \mathbb{E}_x \left[ \frac{1}{T} \sum_{t=1}^{T} \frac{1}{|\mathcal{K}_t|} \sum_{i \in \mathcal{K}_t} \mathcal{D}_{\text{KL}} \left( p_\theta \left( y^i | y_{\text{student},t}, x \right) || p_\theta \left( y^i | y_{\text{teacher},t}, x \right) \right) \right]. \end{aligned} \tag{12}$$

Additionally, we find that the quality of the student trajectory $Y$ influences the final performance (see Section 4.4 and Section E.1). Therefore, for each prompt $x$, we keep sampling trajectories until a correct final answer $y_0$ occurs, or the sampling iteration number meets a threshold (similar to pass@$k$, and we set $k = 8$ by default)[2]. Note that this sampling strategy shares the same computation overhead as RLVR (group $k$ rollouts) for each training step. Following [Zhao et al., 2026], we apply pointwise KL clipping and the fix teacher strategy, as detailed in Section C. Additional implementation details are also provided in Section C, including an important engineering technique preventing out of memory by concatenating step-level inputs, motivated by [Wang et al., 2025].

Crucially, we conclude this section by highlighting the fundamental distinctions between our approach and existing self-distillation approaches for dLLMs, such as d3llm [Qian et al., 2026] and Cd4lm [Liang et al., 2026], which also construct a self-teacher by partially revealing answers. First and foremost, the revealed answers in our approach are "self-experience" generated on-policy by the student itself, whereas theirs are from the ground-truth of static datasets. Second, while we leverage step-level divergence supervision across an entire on-policy generation trajectory, they employ a single forward pass like a 'one-step" fake trajectory to provide supervision. These critical differences define d-OPSD as an on-policy distillation approach providing dense supervision for every denoising steps across the entire trajectory, whereas their approaches remain fundamentally off-policy closely related to SFT.

# 4 Experiments

In this section, we first address a foundational prerequisite with a toy verification:

**Question:** Is the self-teacher strong enough to guide self-distillation?
**Answer:** Yes. The self-teacher is capable enough that the correct answer can be resumed using our teacher construction. See Section 4.1.

We then conduct comprehensive experiments to answer the following core questions:

**Question:** How does OPSD compare to SFT and RLVR in reasoning performance and sample efficiency?
**Answer:** It outperforms or matches SFT and RLVR baselines in reasoning performance, while demonstrating vastly superior sample efficiency. See Section 4.2.

**Question:** How does the self-teacher construction in d-OPSD compare to the AR-style counterpart (Figure 2)?
**Answer:** Our approach significantly outperforms the AR counterpart. The key reason is that our teacher construction introduces more new knowledge (thinking patterns) to transfer to the student. See Section 4.3.

**Question:** What is the impact of different training settings?
**Answer:** We provide comprehensive ablation results on different KL objectives, retaining ratios $\rho_{\text{teacher}}$, top-$k$ subset $\mathcal{K}_t$ selections, sampling strategies, and other training settings. See Section 4.4 and Section E.1.

**Question:** What are the failure modes for d-OPSD?
**Answer:** Similar to RLVR, OPSD is susceptible to policy collapse after reaching peak performance. See Section 4.5.

## 4.1 Experimental Setup & Toy Verification

**Models and Tasks.** We employ LLaDA-8B-Instruct [Nie et al., 2025], a state-of-the-art dLLM that has not undergone post-training, as our base model [3]. We conduct experiments across four reasoning tasks spanning two categories: mathematical reasoning and planning. The mathematical reasoning

[2]Even with $k = 1$, our approach still surpasses the RLVR baseline which uses group $k = 8$ rollouts, see Section 4.4.

[3]We did not use Dream [Ye et al., 2025] because its output format is highly inconsistent, which causes severe instability across RLVR baselines. This limitation is also marked by [Pan et al., 2025].

Table 1: Reasoning performance comparison across four reasoning tasks. Results of diffu-GRPO and the SFT varient are sourced from the original paper [Zhao et al., 2025]. Results of VRPO, d3LLM and the base model are evaluated using their open-sourced models. d-OPSD consistently outperforms or matches SFT and RLVR baselines.

| Model / Seq Len | GSM8K | | MATH500 | | Countdown | | Sudoku | |
|---|---|---|---|---|---|---|---|---|
| | 256 | 512 | 256 | 512 | 128 | 256 | 128 | 256 |
| LLaDA-8B-Instruct | 76.0 | 79.5 | 31.8 | 36.2 | 20.3 | 19.1 | 11.5 | 6.9 |
| **SFT** | | | | | | | | |
| SFT Varient [Zhao et al., 2025] | 78.8 | 81.1 | 32.6 | 34.8 | 20.3 | 14.5 | 16.5 | 8.5 |
| d3LLM [Qian et al., 2026] | 72.7 | 76.4 | 30.6 | 37.0 | 36.7 | 12.5 | 9.1 | 6.6 |
| **RLVR** | | | | | | | | |
| diffu-GRPO [Zhao et al., 2025] | 79.8 | 81.9 | 37.2 | 39.2 | 33.2 | 31.3 | 18.4 | 12.9 |
| VRPO [Zhu et al., 2025] | 80.1 | 81.5 | 35.6 | 34.8 | 21.9 | 21.1 | 12.8 | 9.6 |
| **OPSD** | | | | | | | | |
| **d-OPSD (Ours)** | 81.0 | 82.2 | 37.2 | 37.8 | 37.9 | 32.3 | 23.9 | 20.6 |

tasks include GSM8K [Cobbe et al., 2021] and MATH500 [Lightman et al., 2023]. The planning tasks include $4x4$ Sudoku puzzles, which require constraint satisfaction to fill a grid with numbers, and Countdown (3 numbers), where models must reach a target number using basic arithmetic operations on a given set of integers. All datasets configurations remain consistent with the RLVR baseline, diffu-GRPO [Zhao et al., 2025].

**Baselines.** We compare against two categories of post-training methods: RLVR and SFT. RLVR baselines include diffu-GRPO [Zhao et al., 2025] and VRPO [Zhu et al., 2025]. For SFT, we compare against the SFT variant from [Zhao et al., 2025] and the existing off-policy self-distillation approach, d3LLM [Qian et al., 2026].

Table 2: Sample efficiency comparison between the RLVR baseline and our approach. The optimization steps for diffu-GRPO are sourced from the original paper [Zhao et al., 2025].

| **Method / Step** | **GSM8K** | **Math500** | **Countdown** | **Sudoku** |
|---|---|---|---|---|
| diffu-GRPO | 7700 | 6600 | 5000 | 3800 |
| **d-OPSD (Ours)** | 425 | 100 | 175 | 425 |

**Training Details.** Following [Zhao et al., 2026], we fix the teacher policy to the initial policy to stabilize training. We use full-vocabulary logit distillation with LoRA [Hu et al., 2022]. The default distribution divergence measure is reverse KL. The generation length and retaining ratio $\rho_{\text{teacher}}$ are set to 256 and 0.25, respectively. Additional training details are provided in Section D.1.

Table 3: Toy Verification. The correct answer can be resumed from the self-teacher construction.

| **Method / Tasks** | **GSM8K** | **Math500** | **Countdown** | **Sudoku** |
|---|---|---|---|---|
| Pass@1 (Student) | 81.3 | 36.8 | 18.2 | 7.2 |
| Pass@8 | 95.5 | 53.6 | 59.2 | 27.7 |
| Self-Teacher | | | | |
| $\rho_{\text{teacher}} = 0.10$ | 85.6 | 40.8 | 29.6 | 11.0 |
| $\rho_{\text{teacher}} = 0.25$ | 93.4 | 46.2 | 45.4 | 13.5 |
| $\rho_{\text{teacher}} = 0.50$ | 94.8 | 47.4 | 48.2 | 15.4 |

**Evaluation Details.** We evaluated every 25 steps before step 501 and report the best results. For mathematical reasoning tasks, we evaluate model performance using generation lengths of 512 and 256. For planning tasks, we evaluate at 128 and 256. This distinction is made because longer generation lengths improve performance in mathematical reasoning tasks but degrade it in planning tasks (see Table 1). We utilize the block diffusion strategy [Arriola et al., 2025] with a block length of 32. Denoising steps are configured as half of the generation length.

**Toy Verification.** A critical question that must be answered before the full experiment is whether the self-teacher is strong enough to guide distillation. To verify this, we randomly sampled 500 questions from each task's training set, obtained generations from the base model, constructed self-teacher inputs (using Pass@8) as described in Section 3.3 under different retaining ratios $\rho_{\text{teacher}}$, and finally re-generated responses conditioned on these self-teacher inputs. As shown in Table 3, even with a moderate $\rho_{\text{teacher}} = 0.10$, the self-teacher significantly outperforms the student. At

higher $\rho_{\text{teacher}}$, the self-teacher performance nearly matches its origin (Pass@8). This toy experiment successfully validates that our self-teacher can resume correct answers and guide high-quality distillation. Additional details and examples of this toy experiment are provided in Section D.2.

### 4.2 Main Results

Table 1 presents a comprehensive performance comparison between SFT, RLVR, and our approach. d-OPSD consistently outperforms or matches SFT and RLVR baselines, achieving state-of-the-art performance in most settings and showcasing significant improvements over the base models. Table 2 and Figure 1 detail the sample efficiency comparison between the RLVR baseline and our approach. d-OPSD demonstrates vastly superior sample efficiency, converging in only around $10\%$ of the optimization steps (number of gradient updates) required by RLVR. Note that the pass@$k$ sampling strategy we use in Section 3.3 shares the same computation overhead as RLVR (group $k$ rollouts) for each optimization step. Consistent with [Lu and Lab, 2025, Zhao et al., 2026], we attribute OPSD's superior sample efficiency to the dense supervision provided by the teacher distribution. These results underscore our approach's promising reasoning performance and sample efficiency.

### 4.3 Comparison with AR-style OPSD: Unlocking New Knowledge

A pivotal design choice in our approach is the specific self-teacher construction tailored for dLLMs (Section 3.1). It is imperative to evaluate how this formulation compares to the AR-style construction shown in Figure 2. To this end, we conducted an additional AR-style baseline strictly following [Zhao et al., 2026], which appends the reference solution to the prompt as a prefix conditioning to provide privileged information to the teacher, while keeping our step-level divergence supervision (Section 3.2) constant. Table 4 [4] reports the performance comparison results. Our approach consistently outperforms the AR-style counterpart, highlighting the critical importance of our specific self-teacher construction.

Table 4: Reasoning performance Comparison between AR-style OPSD and our approach. Generation length is 256. Our teacher construction outperforms the AR-style baseline.

| **Method / Tasks** | **GSM8K** | **Math500** |
|---|---|---|
| LLaDA-8B-Instruct | 76.0 | 31.8 |
| AR-style OPSD | 78.4 | 33.4 |
| **d-OPSD (Ours)** | 81.0 | 37.2 |

We further investigate the mechanism behind this performance gap. We define the metric of *Overlap Top-*$K_t$. At each denoising step $t$, it measures the proportion of tokens that appear simultaneously in both the student's and teacher's Top-$K$ vocabulary distributions over the top-$k$ subset $\mathcal{K}_t$ masked positions. Note that Top-$K$ and top-$k$ have different meanings. Top-$K$ refers to comparing the distribution over the vocabulary at a specific token position, while top-$k$ refers to the most confident tokens in the currently masked positions (Section 3.2). Formally, Overlap Top-$K_t$ can be expressed as:

$$\mathcal{M}_{\text{overlap},K,t} = \frac{1}{|\mathcal{K}_t|} \sum_{i \in \mathcal{K}_t} \left[ \frac{|\mathcal{P}^{i,\text{Top-}K}_{\text{student},t} \cap \mathcal{P}^{i,\text{Top-}K}_{\text{teacher},t}|}{K} \right], \tag{13}$$

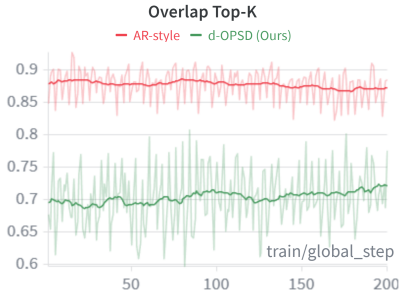


Figure 3: The Overlap Top-K comparison between d-OPSD and the AR-style counterpart.

where $\mathcal{P}^{i,\text{Top-}K}_t$ is the Top-$K$ distribution over the vocabulary at token position $i$, derived from Equation (9). As shown in Figure 3, the Overlap Top-$K_t$ for AR-style OPSD is extremely high, nearly to 1, indicating that appending a reference solution fails to bring new knowledge or thinking patterns to the teacher for the student to learn. Conversely, the Overlap Top-$K_t$ for d-OPSD lies in a suitable range, providing more new knowledge that can be transferred from teacher to student. $K$ is set to $K = 20$ in practice.

Table 5: Reasoning performance comparison of divergence objectives.

| **Method / Tasks** | **GSM8K** |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| Forward KL | 77.9 |
| Reverse KL (default) | 81.0 |

[4] Following [Zhao et al., 2026], the reference solution is the reasoning trajectory obtained directly from the dataset. Therefore, we did not conduct experiments on Countdown and Sudoku, as they consist of only questions and pure ground truths without any reasoning traces.

### 4.4 Ablation Studies

Additional ablation studies are provided in section E.

**Divergence Objective.** We compare reverse KL (default) and forward KL in Table 5. Reverse KL clearly outperforms forward KL. We attribute this to the model-seeking behavior of reverse KL [Agarwal et al., 2024], which is more robust compared to the model-covering behavior of forward KL.

Table 6: Reasoning performance comparison of retaining ratios.

| Method / Tasks | GSM8K |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| diffu-GRPO | 79.8 |
| d-OPSD | |
| $\rho_{\text{teacher}} = 0.10$ | 80.5 |
| $\rho_{\text{teacher}} = 0.25$ (default) | 81.0 |
| $\rho_{\text{teacher}} = 0.50$ | 79.8 |

**Retaining Ratio.** We observe that different retaining ratios $\rho_{\text{teacher}}$ have moderate influences on overall performance. As shown in Table 6, all configurations improve over the base model and surpass the RLVR baseline. Interestingly, $\rho_{\text{teacher}} = 0.10$ yields better results than $\rho_{\text{teacher}} = 0.50$, despite it is a weaker teacher as shown in Table 3. This suggests that while a accurate teacher is beneficial, the distillation effectiveness is not only decided by the teacher performance.

**top-$k$ subset $\mathcal{K}_t$ Selection.** As noted in section 3.2, $\mathcal{K}_t$ can be selected using either the student distribution or teacher distribution. Table 7 compares these two choice. Deriving $\mathcal{K}_t$ from the teacher distribution yields higher performance, as it forces the student to align with the most confident distributions by the teacher policy, providing a stronger learning signal.

Table 7: Reasoning performance comparison of $\mathcal{K}_t$ selections.

| Method / Tasks | GSM8K |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| From Student | 78.6 |
| From Teacher (default) | 81.0 |

**Pass@$k$.** As noted in Section 3.3, we employ a sampling strategy akin to pass@$k$, keeping sampling trajectories until a correct answer occurs within $k$ iterations.

Table 8 evaluates the impact of varying $k$. Although $k = 1$ slightly degrades reasoning performance compared to $k = 8$, it still surpasses the RLVR baseline with a even greater sample efficiency than $k = 8$.

Table 8: Reasoning performance comparison of sampling strategies.

| Method / Tasks | GSM8K | Countdown |
|---|---|---|
| LLaDA-8B-Instruct | 76.0 | 20.3 |
| diffu-GRPO | 79.8 | 33.2 |
| d-OPSD | | |
| $k = 1$ | 80.4 | 34.0 |
| $k = 8$ (default) | 81.0 | 37.9 |

**Per-token Pointwise Clipping.** As noted in Section C.1, we adopt a pointwise clipping strategy following [Zhao et al., 2026]. Table 9 shows that pointwise clipping substantially improves the performance of d-OPSD. More importantly, we observe that clipping stabilizes training in most settings, which explains the performance gap. In contrast, the none-clipping variant starts to collapse around step 150, with performance finally dropping to 69.37 by step 500. The clipping threshold is set to 0.05 in practice.

### 4.5 Failure Modes

We wish to transparently share a failure mode observed with our current approach. Although it is highly effective in both reasoning performance and sample efficiency, we find that similar to RLVR, OPSD in some settings is prone to policy collapse after achieving peak performance.

Table 9: Reasoning performance comparison of clipping.

| Method / Tasks | GSM8K |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| No-clip | 77.0 |
| Clip (default) | 81.0 |

As shown in Figure 12, training sometimes degrade catastrophically. We noticed that the same phenomena is commonly observed in RLVR [Deng et al., 2025, Bai et al., 2025]. We hypothesize that this collapse may stem from the model-seeking behavior [Agarwal et al., 2024] becoming overly narrow, prevent from further learning.

## 5 Related Works

Additional relate works are provided in Section A.2.

**On-policy Distillation.** Knowledge distillation [Hinton et al., 2015] transfers knowledge from a large teacher model to a smaller student model by training on the teacher's soft output distributions.

Kim and Rush [2016], Jiao et al. [2020], Wang et al. [2020] leveraged it to sequence-level distillation, establishing the dominant off-policy distillation approaches. [Gu et al., 2024, Agarwal et al., 2024, Lu and Lab, 2025, Yang et al., 2026] extended it to OPD, addressing the exposure bias [Bengio et al., 2015] mismatch by shifting the training distribution to the student's own generations.

## 6 Conclusion

This work presents d-OPSD, the first on-policy self-distillation approach for dLLMs. It is specifically tailored to align with on-policy and dLLMs nature. We propose a novel self-teacher construction that utilizes the model's own self-generated answers as suffix conditioning for privileged information, effectively guiding the student to learn from its on-policy "self-future experience". Furthermore, we shift the dense divergence supervision from the token-level to step-level, perfectly matching the iterative mechanics of dLLMs. Future work will explore advanced techniques to further stabilize and enhance the OPSD post-training of dLLMs.

## References


Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.

Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huanang Gao, Wenkai Yang, Zhiyuan Liu, et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. *Advances in neural information processing systems*, 28, 2015.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *arXiv preprint arXiv:2406.03736*, 2024.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.

Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, et al. Llada2. 0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.

Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv e-prints*, pages arXiv–2506, 2025.

Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, et al. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*, 2025.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

Xiaohang Tang, Rares Dolga, Sangwoong Yoon, and Ilija Bogunovic. wd1: Weighted policy optimization for reasoning in diffusion language models. *arXiv preprint arXiv:2507.08838*, 2025.

Shaoan Xie, Lingjing Kong, Xiangchen Song, Xinshuai Dong, Guangyi Chen, Eric P Xing, and Kun Zhang. Step-aware policy optimization for reasoning in diffusion large language models. *arXiv preprint arXiv:2510.01544*, 2025.

Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025.

Yu-Yang Qian, Junda Su, Lanxiang Hu, Peiyuan Zhang, Zhijie Deng, Peng Zhao, and Hao Zhang. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation. *arXiv preprint arXiv:2601.07568*, 2026.

Yihao Liang, Ze Wang, Hao Chen, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Emad Barsoum, Zicheng Liu, and Niraj K Jha. Cd4lm: Consistency distillation and adaptive decoding for diffusion language models. *arXiv preprint arXiv:2601.02236*, 2026.

Leyi Pan, Shuchang Tao, Yunpeng Zhai, Zheyu Fu, Liancheng Fang, Minghua He, Lingzhe Zhang, Zhaoyang Liu, Bolin Ding, Aiwei Liu, et al. d-treerpo: Towards more reliable policy optimization for diffusion language models. *arXiv preprint arXiv:2512.09675*, 2025.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The twelfth international conference on learning representations*, 2023.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.

Wenlong Deng, Yushu Li, Boying Gong, Yi Ren, Christos Thrampoulidis, and Xiaoxiao Li. On grpo collapse in search-r1: The lazy likelihood-displacement death spiral. *arXiv preprint arXiv:2512.04220*, 2025.

Bizhe Bai, Hongming Wu, Peng Ye, and Tao Chen. M-grpo: Stabilizing self-supervised reinforcement learning for large language models with momentum-anchored policy optimization. *arXiv preprint arXiv:2512.13070*, 2025.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1317–1327, 2016.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. In *Findings of the association for computational linguistics: EMNLP 2020*, pages 4163–4174, 2020.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems*, 33:5776–5788, 2020.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The twelfth international conference on learning representations*, 2024.

Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.

Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. Ssd-lm: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596, 2023.

Nima Fathi, Torsten Scholak, and Pierre-André Noël. Unifying autoregressive and diffusion-based sequence generation. *arXiv preprint arXiv:2504.06416*, 2025.

Zemin Huang, Zhiyang Chen, Zijun Wang, Tiancheng Li, and Guo-Jun Qi. Reinforcing the diffusion chain of lateral thought with diffusion language models. *arXiv preprint arXiv:2505.10446*, 2025.

Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation. *arXiv preprint arXiv:2506.20639*, 2025.

Kevin Rojas, Jiahe Lin, Kashif Rasul, Anderson Schneider, Yuriy Nevmyvaka, Molei Tao, and Wei Deng. Improving reasoning for diffusion language models via group diffusion policy optimization. *arXiv preprint arXiv:2510.08554*, 2025.

Jingyang Ou, Jiaqi Han, Minkai Xu, Shaoxuan Xu, Jianwen Xie, Stefano Ermon, Yi Wu, and Chongxuan Li. Principled rl for diffusion llms emerges from a sequence-level perspective. *arXiv preprint arXiv:2512.03759*, 2025.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL `https://github.com/huggingface/trl`.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

# A Additional Preliminaries and Related Works

## A.1 Additional Preliminaries

**Block-diffusion.** In practice, the block-diffusion inference strategy [Han et al., 2023, Arriola et al., 2025, Fathi et al., 2025] is commonly used in current dLLMs. This hybrid approach partitions a response $y$ into $B$ contiguous, non-overlapping blocks $\{\text{block}_1, \text{block}_2, \cdots, \text{block}_B\}$, with each block containing $L' = \frac{L}{B}$ tokens. The inference is purely AR at the block level while being purely diffusion-style within each block, where the next block starts to decode only when the last block gets fully decoded.

## A.2 Additional Related Works

**Reinforcement Learning for dLLMs.** Reinforcement learning (RL) has emerged as a critical post-training technique for enhancing the reasoning capabilities of dLLMs. Most existing works [Zhao et al., 2025, Huang et al., 2025, Tang et al., 2025, Gong et al., 2025, Rojas et al., 2025, Ou et al., 2025] directly apply GRPO to dLLMs, using either one-step estimation or the ELBO to estimate the log-probability in GRPO. However, most of them suffer from the fundamental challenges of RLVR: the heavy computation overhead and the bottleneck of spare rewards.

# B Self-teacher Construction Illustrations

Here we provide an example of how our self-teacher construction (Section 3.1) works, with a question sampled from GSM8K training set. For brevity, we omit some `mask` and "end-of-text" tokens.

The question is:

*Question: Bob was creating a math test for an online platform. He created 13 questions in the first hour. Bob then doubled his rate for the second hour, and doubled his second hour rate for the third hour. How many questions did Bob create in the three hours?*

Figure 4: A question from GSM8K training set.

First, we sample an on-policy trajectory [5] from the student model and obtain the final clean answer as the self-generated future:

*On-policy Final Answer (Self-generated Future)*

```
<reasoning>
To determine the total number of questions Bob created in three hours, we need to calculate
the number of questions he created each hour and then sum them up.

1. In the first hour, Bob created 13 questions.
2. In the second hour, Bob doubled his rate from the first hour. Therefore, he created
     \[
     13 \times 2 = 26
     \] questions.
3. In the third hour, Bob doubled his rate from the second hour. Therefore, he created
     \[
     26 \times 2 = 52
      \] questions.

Now, we add the number of questions created each hour to find the total:
    \[
    13 + 26 + 52 = 91
     \]

Thus, Bob created a total of 91 questions in the three hours.
</reasoning>
<answer>
\\boxed{91}
<answer><|eot_id|><|endoftext|>…
```

Figure 5: The self-generated future answer.

[5] Using pass@$k$, it keeps sampling until a correct final answer appears or it reaches the iteration threshold.

At denoising step $t = 20$, we have the student decoding status as follows:

**Student Decoding**

```
<reasoning>
To determine the total number of questions Bob created in three hours, we need to calculate
the number of questions he created each hour and then sum them up.

1.<|mask|><|mask|><|mask|>...
```

Figure 6: Current student decoding status.

We then construct the self-teacher at step $t = 20$ as follows:

**Self-teacher Construciton**

```
<reasoning>
To determine the total number of questions Bob created in three hours, we need to calculate
the number of questions he created each hour and then sum them up.

1.<|mask|>...<|mask|> hour<|mask|><|mask|>,<|mask|> created
\[<|mask|>...<|mask|>2<|mask|><|mask|> questions<|mask|>...<|mask|> the<|mask|>...<|mask|>
his rate from<|mask|> second<|mask|><|mask|> Therefore<|mask|>
he<|mask|>...<|mask|>2<|mask|><|mask|><|mask|><|mask|>Now<|mask|><|mask|> add<|mask|><|mask|>
of questions<|mask|>...<|mask|> find<|mask|><|mask|>:
<|mask|>Now<|mask|><|mask|> add<|mask|><|mask|> of questions<|mask|>...<|mask|>
find<|mask|><|mask|>:
<|mask|>...<|mask|> +<|mask|>2<|mask|>...<|mask|>2 =<|mask|>91<|mask|>...<|mask|>,
Bob<|mask|><|mask|><|mask|> of<|mask|><|mask|>1<|mask|><|mask|><|mask|>
three<|mask|>...<|mask|>boxed<|mask|>...<|mask|>
<|mask|><|mask|><|mask|><|eot_id|><|mask|>...<|mask|><|endoftext|><|mask|>...<|mask|><|endoft
ext|><|mask|><|endoftext|><|endoftext|><|mask|>...<|mask|><|endoftext|><|mask|><|mask|><|endo
ftext|><|mask|><|mask|><|mask|><|endoftext|><|mask|>...<|mask|><|endoftext|><|endoftext|><|ma
sk|><|mask|><|mask|><|endoftext|><|mask|><|mask|>
```

keep same with student decoding  retain part answers from self-future

Figure 7: Self-teacher construction at $t = 20$.

For comparison, we also illustrate the AR-style construction, which appends a reference solution to the prompt, as shown in Figure 8.

# C Additional Implementation Details

## C.1 Per-Token pointwise clipping

Following [Zhao et al., 2026], we apply pointwise clipping to the vocabulary level divergence contributions. The reason is that token-level divergence is highly skewed across vocabulary entries, and our ablation study in Section 4.4 empirically validates that pointwise clipping stabilizes training and leads to better performance.

## C.2 Inputs Concatenation

DLLMs generate responses by an iterative denoising process, where each iteration requires full-attention over all token positions. Consequently, computing the loss objective in Equation (12) can easily lead to out of memory, as the full attention gradient maps over all token positions across every steps need to be stored, until a trajectory is fully decoded. To address this issue, we leverage a engineering technique, concatenating all inputs across every steps of a trajectory into an entire batch. Specifically, assume that the student decoding status is a tensor of shape (bsz, seq-length). Instead of feeding it into the model to compute the corresponding term in Equation (12), we concatenate

*Student Prompt*

```
You are a math expert. You will be given a question to solve. Solve it step by step. Wrap the
final answer in a \\boxed{}.
Respond in the following format:
<reasoning>
Your reasoning here
</reasoning>
<answer>
\\boxed{...}
</answer>

Bob was creating a math test for an online platform. He created 13 questions in the first
hour. Bob then doubled his rate for the second hour, and doubled his second hour rate for the
third hour. How many questions did Bob create in the three hours?
```

*Teacher Prompt*

```
You are a math expert. You will be given a question to solve. Solve it step by step. Wrap the
final answer in a \\boxed{}.
Respond in the following format:
<reasoning>
Your reasoning here
</reasoning>
<answer>
\\boxed{...}
</answer>

Bob was creating a math test for an online platform. He created 13 questions in the first
hour. Bob then doubled his rate for the second hour, and doubled his second hour rate for the
third hour. How many questions did Bob create in the three hours?

Here is a reference solution:
First hour: <<13=13>>13
Second hour:13*(2)=26
Third hour: 26*(2)=52
Total:13+26+52=<<13+26+52=91>>91 questions

After understanding the reference solution, please try to solve this problem using your own
approach below:
```

Figure 8: AR-style teacher construction.

all status tensors across all steps of this trajectory to form a "batch" tensor of shape (bsz$\times$steps, seq-length). Since all inputs share the same model, the gradient remains constant for each input and no longer needs to be stored as previously.

### C.3 Compute only on Correct Generations

By default, we compute the loss objective Equation (12) only on correct generations [6]. Although computing on all generations also improves the model's reasoning performance, our default setting achieves superior results. Detailed experimental results are provided in Section E.1.

## D Additional Experiment Details

### D.1 Training Details

We used the TRL library [von Werra et al., 2020] to implement d-OPSD. We employed Low-Rank Adaptation (LoRA) with a rank of $r = 128$ and scaling factor $\alpha = 64$. Training was conducted on $4$ NVIDIA GPUs, with a learning rate of $5 \times 10^{-6}$, accumulation steps of $1$, the AdamW optimizer

[6] For Sudoku task, there are no "right" or "wrong" answers because it gives a score in $[0, 1]$. Therefore, we set an threshold to decide if the generation should be include in loss computation. In practice, the threshold is set to $0.25$.

[Loshchilov and Hutter, 2017], and Flash Attention 2 [Dao, 2023]. The RLVR baseline diffu-GRPO [Zhao et al., 2025] in Figure 1 was reproduced on 8 NVIDIA GPUs, following its default settings.

### D.2 Toy Experiment Details and Examples

The generation length is 256 for all tasks. After applying the self-teacher construction, the number of remaining `mask` tokens becomes smaller than the generation length 256. We keep constant that unmasking 2 tokens in each step with a block length of 32.

One important point to note is that, to prevent the risk of leaking the final answer (e.g., the final answer between <answer><answer> is retained in the self-teacher construction), everytime we move to a new block, we clear all unmasked tokens in this block, leaving the new block entirely filled with only `mask` tokens.

We provide an example from GSM8K training set. The question is:

*Question: Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?*

Figure 9: A question from GSM8K training set.

First, we sample a generation [7] from the student model and obtain the final clean answer:

*On-policy Final Answer (Self-generated Future)*

```
<reasoning>
To determine the total number of clips Natalia sold in April and May, we need to calculate
the number of clips sold in each month and then sum them up.

1. Natalia sold 48 clips in April.
2. She sold half as many clips in May as she did in April. Therefore, the number of clips
sold in May is
      \[
      \frac{48}{2} = 24
      \]

3. To find the total number of clips sold in both months, we add the number of clips sold in
April to the number of clips sold in May:
    \[
    48 + 24 = 72
    \]

Thus, Natalia sold a total of 72 clips in April and May.
</reasoning>
<answer>
\\boxed{72}
<answer><|eot_id|><|endoftext|>…
```

Figure 10: The self-generated answer.

We then construct self-teacher by partially revealing the final generation, as shown in Figure 11.

## E Additional Experiment Results

### E.1 Additional Ablation Studies

**Fixing the Teacher.** We find that fixing the teacher model leads to greater performance gains, as shown in Table 10. Notably, even when the teacher is not fixed, d-OPSD's reasoning performance nearly matches the RLVR baselines, further demonstrating its effectiveness.

Table 10: Reasoning performance comparison of teacher fixing.

| Method / Tasks | GSM8K |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| diffu-GRPO | 79.8 |
| d-OPSD | |
| Unfix teacher | 79.7 |
| Fix teacher (default) | 81.0 |

[7] Using pass@$k$, it keeps sampling until a correct final answer appears or it reaches the iteration threshold.

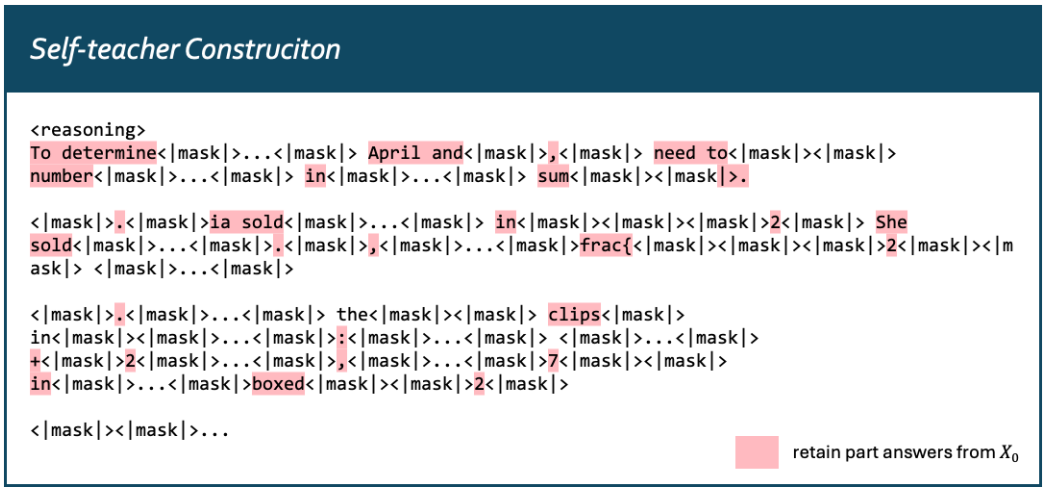


Figure 11: Self-teacher in the toy experiment.

**Compute only on Correct Generations**

As shown in Table 11, computing the loss on all trajectories leads to a slight performance degradation. Nevertheless, it still outperforms the RLVR baseline.

Table 11: Reasoning performance comparison.

| Method / Tasks | GSM8K |
|---|---|
| LLaDA-8B-Instruct | 76.0 |
| diffu-GRPO | 79.8 |
| d-OPSD | |
| All | 80.3 |
| Correct only (default) | 81.0 |

### E.2 Qualitative Examples on GSM8k

We provide a qualitative example from GSM8k testing set, where the RLVR model gives an incorrect answer while our approach yields the correct one, as shown in Figure 13.

### E.3 Failure Mode

Figure 12 presents the failure mode mentioned in Section 4.5.

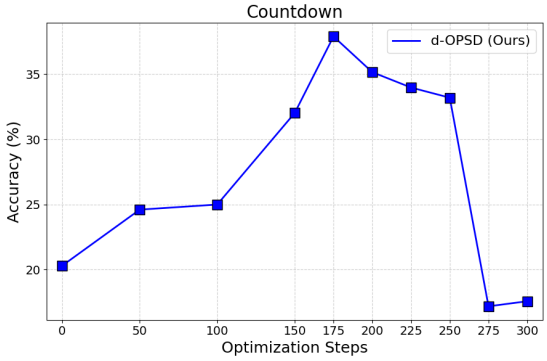


Figure 12: Failure Mode of collapse.

*Question: In a student council election, candidate A got 20% of the votes while candidate B got 50% more than candidate A's votes. The rest of the votes was given to candidate C. If there were 100 voters, how many votes did candidate C get?*

*Ground Truth: 50*

*Model: d-OPSD (Ours)*

```
<reasoning>
1. Candidate A got 20% of the votes, which is
    \\[
    0.20 \\times 100 = 20
    \\] votes.
2. Candidate B got 50% more than candidate A's votes, which is
    \\[
    0.50 \\times 20 = 30
    \\] votes.
3. The total votes for candidates A and B is
    \\[
    20 + 30 = 50
    \\] votes.
4. Therefore, the votes given to candidate C is
    \\[
    100 - 50 = 50
    \\] votes.
</reasoning>
<answer>
\\boxed{50}
</answer><|eot_id|><|endoftext|>…
```

*Model: RLVR (diffu-GRPO)*

```
<reasoning>
To determine how many votes candidate C received, we need to follow these steps:
1. Calculate the number of votes candidate A received.
2. Calculate the number of votes candidate B received.
3. Subtract the votes received by candidates A and B from the total number of votes to
find the votes received by candidate C.

First, calculate the total number of votes candidate A received:
    \\[
    \\text{Votes for A} = 20\\% \\text{ of } 100 = 0.20 \\times 100 = 20
    \\]
Next, calculate the number of votes candidate B received:
    \\[
    \\text{Votes for B} = 50\\% \\text{ of } 20 = 0.50 \\times 20 = 10 \\]

Now, subtract the votes received by candidates A and B from the total number of votes to
find the votes received by candidate C:
    \\[
    \\text{Votes for C} = 100 - 20 - 10 = 70
    \\]

</reasoning>
<answer>
\\boxed{70}
</answer><|eot_id|><|endoftext|>
```

Figure 13: Qualitative Examples on GSM8k